Multi-Agent Debugging — Worked Example: Solution

Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Hints

Story — `multi_agent_debugging_practice.py`: multi-agent bugs tend to live *between* agents, not inside any one of them — a mismatched route name, a state field one agent expects and another never sets, a tool description two agents both rely on. Each fix here comes with a test that checks that exact boundary directly, using stand-in agents and, except for Round 3, no model call at all. **If not:** every agent would keep passing its own isolated tests while the team as a whole kept failing in production.

Every fix and test below goes in `practice/multi_agent_debugging_practice.py`, and runs with `pytest multi_agent_debugging_practice.py -v` from inside `practice/`. Only Round 3's test calls the real model (`OPENAI_API_KEY` in your `.env`).

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent
- Real cause vs. symptom fix

Round: Basic

Real cause: a plain typo — `"Reserach"` instead of `"research_agent"`. `Command(goto=...)` has no fuzzy matching, and a `goto` to a name that isn't a node doesn't raise: LangGraph logs a warning ("wrote to unknown channel branch:to:Reserach, ignoring it") and the run simply ends. That's why nothing crashed — and why the warning line in the log is the only clue.

Story: a failure that raises no exception at all is the hardest kind to notice, because nothing alerts you to it. This round trains actually reading warning-level log lines, not just scanning for errors, and keeping node names defined in one place so a typo between two references becomes structurally impossible. **If not:** runs would keep silently ending early and empty, and every error-monitoring alert would stay quiet the whole time.

The fix:

```
# why: every node name written ONCE — add_node and every route
# use these, so a typo is a NameError, not a silent early stop
RESEARCH = "research_agent"
ANALYSIS = "analysis_agent"
WRITER = "writer_agent"
NODE_NAMES = [RESEARCH, ANALYSIS, WRITER]

def decide_next_agent(state):
    if not state.get("research_findings"):
        return RESEARCH
    if not state.get("analysis"):
        return ANALYSIS
    return WRITER
```

The graph uses the same names: `graph.add_node(RESEARCH, research_node)`, and so on.

Test that would have caught it:

```
def test_supervisor_only_routes_to_real_node_names():
    states = [
        {},
        {"research_findings": "found it"},
        {"research_findings": "found it", "analysis": "means this"},
    ]
    for state in states:
        # how: every possible route must be one of the graph's nodes
        assert decide_next_agent(state) in NODE_NAMES
```

The test walks the supervisor through each stage and checks every name it can return — the typo'd name fails here, loudly, instead of quietly in production.

Round: Intermediate

Real cause: `research_node` writes to a key (`finding`) that `SharedState` never declared — the schema has `research_findings`. LangGraph quietly ignores an update to an undeclared key, so the Research agent "succeeds" and the Analysis agent correctly reads an empty `research_findings`. Each is right about its own narrow view; the mismatch exists only between them.

Story: the log shows the data present one line above exactly where it goes missing — the gap is the hand-off between two nodes itself. This round trains writing a test that checks precisely which state keys a node actually writes, not just that it runs without crashing. **If not:** the pipeline would keep

finishing "successfully" with weak, generic answers, and no error would ever surface anywhere to say why.

The fix:

```
from typing import TypedDict
from langgraph.types import Command

class SharedState(TypedDict):
    task: str
    research_findings: str
    analysis: str
    draft: str

def run_research_tools(task):
    # stand-in for the Research agent's real tool calls
    return "The refund policy allows returns within 30 days of purchase."

def research_node(state):
    finding = run_research_tools(state["task"])
    # why: write to the field the schema declares — and the one the
    # Analysis agent reads
    return Command(goto="analysis_agent",
                   update={"research_findings": finding})
```

Test that would have caught it:

```
def test_research_node_writes_the_field_analysis_reads():
    result = research_node({"task": "refund policy"})
    assert "research_findings" in result.update
    assert "30 days" in result.update["research_findings"]
```

Checking the *keys* a node writes catches silent naming drift that "did the pipeline finish without an error" never would.

Round: Real-world

Real cause: the two specialist descriptions overlap for any task that needs both finding *and* interpreting — nothing tells the supervisor which one wins, so its choice (an LLM call) follows small wording differences. This is Doc11's "ambiguous routing" edge case: it looks like random model behavior, but it's a gap in how the specialists were described.

Story: five near-identical tasks come back routed two different ways — the pattern seen *across* several runs is the real evidence here, not any single run. This round trains fixing the actual tool or

agent descriptions the model chooses between, and testing with several differently-worded versions of the same task to confirm the fix generalizes. **If not:** you'd blame model randomness, lower the temperature to mask it, and leave the real ambiguity in the descriptions untouched.

The fix:

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

# why: each description says what the agent does AND what it doesn't,
# so for any task only one of them fits
SPECIALISTS = {
    "research_agent": (
        "Finds NEW factual information the system doesn't have yet. "
        "Does not interpret or draw conclusions."
    ),
    "analysis_agent": (
        "Interprets information that has ALREADY been found. "
        "Does not search for new facts."
    ),
}

class RoutingDecision(BaseModel):
    next_agent: str
    reason: str

router_model = ChatOpenAI(
    model="gpt-4o-mini", temperature=0
).with_structured_output(RoutingDecision)

def supervisor_decide(state):
    descriptions = ""
    for name in SPECIALISTS:
        descriptions = descriptions + name + ": " + SPECIALISTS[name] + "\n"
    decision = router_model.invoke(
        "Specialists:\n" + descriptions + "Task: " + state["task"] + "\n"
        "Which specialist should run FIRST?"
    )
    return decision.next_agent
```

For a task that really needs both, the structure handles it: Research runs first, then the supervisor sends its output to Analysis — instead of one decision picking a single winner.

Test that would have caught it:

```
def test_supervisor_routes_the_same_task_the_same_way():
    wordings = [
        "find and interpret last quarter's revenue numbers",
        "look up and interpret last quarter's revenue numbers",
        "get and interpret last quarter's revenue figures",
    ]
    routes = []
    for task in wordings:
        routes.append(supervisor_decide({"task": task}))
    # how: one route for every wording = the ambiguity is gone
    assert len(set(routes)) == 1
```

One wording per task can never show wording sensitivity — the test has to send several phrasings of the same task.

Round: Multi-agent

Real cause: Round 2's bug was "fixed" by *adding* a field (`finding`) instead of renaming, leaving two fields that look alike: `finding` (written by Research, read by Analysis) and `research_findings` (still declared, still read by the Reviewer, written by nobody). Each pair test used whichever field its author picked, so both pairs passed — the disagreement only shows when all 4 agents share one real state.

Story: every pair of agents works fine when tested in isolation; the full team together doesn't. This round trains running the *whole* pipeline once, with stand-in agents, so every hand-off between every pair gets checked against the exact same shared state in one real run. **If not:** each agent's author would keep separately proving their own part correct, while the Reviewer kept rejecting every single draft the full team produced.

The fix:

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

class TeamState(TypedDict):
    task: str
    # why: ONE field name for Research's output — the extra
    # `finding` field is deleted, not kept beside it
    research_findings: str
    draft: str
    review_passed: bool
    revision_count: int
```

```
def research_node(state):
    # stand-in for the real Research agent
    return {"research_findings": "Q1 revenue grew 12% year over year."}

def writer_node(state):
    # stand-in for the real Writer: it uses the research findings
    draft = state["research_findings"] + " Growth came from new products."
    return {"draft": draft,
            "revision_count": state["revision_count"] + 1}

def reviewer_node(state):
    findings = state["research_findings"]
    if findings == "" or findings not in state["draft"]:
        if state["revision_count"] >= 3:
            return Command(goto=END, update={"review_passed": False})
        return Command(goto="writer", update={"review_passed": False})
    return Command(goto=END, update={"review_passed": True})

def build_team_graph():
    graph = StateGraph(TeamState)
    graph.add_node("research", research_node)
    graph.add_node("writer", writer_node)
    graph.add_node("reviewer", reviewer_node)
    graph.add_edge(START, "research")
    graph.add_edge("research", "writer")
    graph.add_edge("writer", "reviewer")
    return graph.compile()
```

Test that would have caught it:

```
def test_full_pipeline_approves_an_accurate_draft():
    start = {"task": "summarize Q1 revenue", "research_findings": "",
             "draft": "", "review_passed": False, "revision_count": 0}
    result = build_team_graph().invoke(start)
    assert result["review_passed"] is True
    assert result["revision_count"] == 1
```

This runs Research, Writer and Reviewer against one shared state, passed hand to hand — the only kind of test that sees two agents reading different fields. With the old two-field setup, it fails at once.

Real cause vs. symptom fix

A symptom fix here would be adding a fallback route when a node name isn't found (Round 1), making the Analysis agent search again when its input is empty (Round 2), setting `temperature=0`

and hoping the routing settles (Round 3), or turning off the Reviewer's check under deadline pressure (Round 4). Each hides one visible failure while the real cause — a mistyped name, two names for one field, overlapping descriptions — stays in place and shows up somewhere else next. In multi-agent systems the real cause almost always sits *between* agents, so the real fix is almost always a shared definition: one list of node names, one field name, one clear boundary between what each agent does.